Energy-Efficient CPU+FPGA-Based CNN Architecture for Intrusion Detection Systems

Lucas A. Maciel, Matheus A. Souza, and Henrique C. Freitas
Pontifical Catholic University of Minas Gerais

***Abstract*—Machine learning applications have their viability strongly linked to the ability of computer architectures to offer high performance and energy efficiency. Although different architectures can offer high computational performance, they may lack energy efficiency, which is crucial for consumer-based applications. For instance, intrusion detection systems can use machine learning techniques to monitor network traffic and identify possible malicious activities. These systems are constantly active on devices such as firewalls, reinforcing the need for energy efficiency, e.g., in smart homes and autonomous vehicles. Field-programmable gate array (FPGA) can offer better energy efficiency than other architectures. Considering that, we designed and evaluated a CPU+FPGA-based convolutional neural network for intrusion detection systems. We deployed our strategy in a heterogeneous CPU (Intel Xeon) + FPGA (Arria 10) platform. Then, we compared the proposed architecture with its respective parallel software version for power, energy, and performance evaluation. The NSL-KDD dataset was used for intrusion detection benchmarking. The energy efficiency results for CNN showed up to 4.5× more operations per watt than its software version.**

Digital Object Identifier 10.1109/MCE.2023.3283730
Date of publication 7 June 2023; date of current version 10 June 2024.

The fast advance of machine learning (ML) techniques in several research areas, such as data classification[1] and intrusion detection,[2] can

produce feasible solutions for complex and heterogeneous consumer-based scenarios such as smart home[3] and autonomous vehicles.[4] Due to the increased traffic generated by traditional devices and Internet-of-Things (IoT),[5] energy-efficient ML architectures are required. Besides, increasing the accuracy of the results and generating them in time-restricted scenarios are two of the leading original goals of ML algorithms. The large volume of data requires attention concerning security, which intensifies investments in monitoring strategies. For instance, intrusion detection systems (IDS) can identify possible attacks, inform their administrators that intrusions may occur, or even prevent the arrival of malicious packages.[6] In this context, IDS can also employ deep learning methods[7] to promote the evaluation of new records according to similarities between the already known data.

One of the most used algorithms for data classification, processing, and image recognition in supervised learning is the convolutional neural network (CNN),[8,9] a deep learning algorithm. It processes data in several layers of connected neurons, where the hidden layers can represent convolutions, pooling, or fully connected processes.

A CNN model generally consists of two stages. The former is *feedforward* for data recognition processes. The latter is *backward* for updating the network weights during training. In addition, it presents components responsible for its operations, such as (i) the feature extractor, which identifies and selects the attributes with the essential information within the *"feature maps"* of the network input data; and (ii) the classifier, which decides the probability of a given entry belonging to the evaluated categories.

CNN applications perform mathematical operations demanding high-performance computing with energy efficiency. Thus, efficient heterogeneous architectures, e.g., central processing unit (CPU) + field-programmable gate array (FPGA) can achieve lower execution time and energy. In this article, our goal is to design and evaluate a CNN for a hybrid CPU (Intel Xeon) + FPGA (Intel Arria 10) platform, using Open Computing Language (OpenCL) in comparison to a CPU-based multithreading OpenMP version. This hybrid processing fits well with next-generation firewalls, which employ deep package inspections. Those firewalls include devices such as the FPGA besides the traditional CPU in their boxes, which can quickly receive updates for novel techniques.[10,11] Also, they can deploy machine learning algorithms to detect zero-day attacks, which standard detection systems cannot identify.[12] Furthermore, using OpenCL as a high-level synthesis (HLS) strategy allows for a more efficient and portable software development process for hardware acceleration, providing a standard programming interface for heterogeneous computing systems.

In summary, our contributions are as follows:

- An energy-efficient strategy to deploy a CNN architecture for network intrusion detection designed for a hybrid multichip package system.
- A more general and easy-to-use high-level synthesis code, based on OpenCL, with a CNN architecture that can be reused for FPGA systems.[a]

This article is organized as follows: Section "Related Work" presents the related work. Next, Section "Proposed CNN Architecture" shows our proposed machine learning architecture. Then, we describe the evaluation methodology in Section "Evaluation Methodology." Finally, we show the results obtained in Section "Results" and our conclusions and future works in Section "Conclusion."

RELATED WORK

Artificial neural networks can be applied in a broad set of applications.[13,14,15] Their use range from security purposes in the Internet of Things (IoT) area[16,17] to autonomous vehicles.[4,18] Regardless of the application or scenario, this section focuses on related works that describe CPU or FPGA-based CNN designs for network intrusion detection.

Due to the increase in Internet traffic, some research works focus on FPGA as an accelerator to improve network intrusion detection (NID) performance. Jeune et al.[19] designed a real-time

[a]https://github.com/cart-pucminas/machine-learning

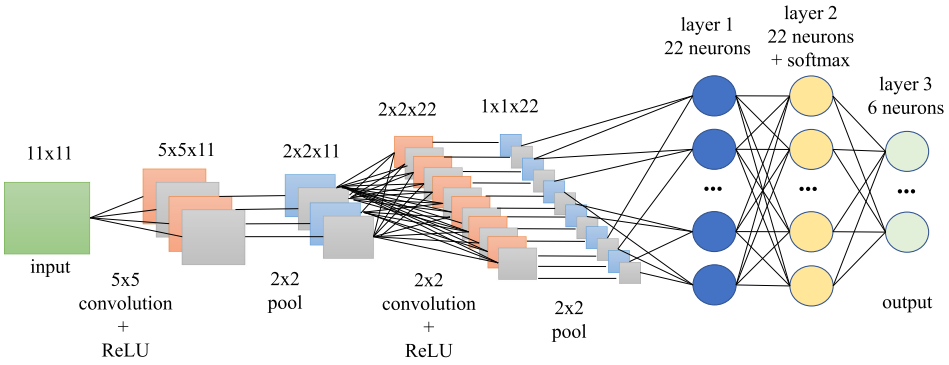

FIGURE 1. CNN architecture diagram.

CNN without reducing performance based on using flow buckets to collect traffic features. The authors also used an FPGA-based design to evaluate the performance with high network speed in further work.[20]

Despite not using the FPGA-based approach, Ding and Zhai[22] proposed an intrusion detection system with a CNN using an NSL-KDD dataset. They evaluated the proposed CNN in comparison to other machine learning algorithms, such as random forest, support vector machine (SVM), deep belief network (DBN), and long short term memory (LSTM). The results pointed out the improvement in the accuracy of detecting intrusions.

Likewise, two research works[21,23] proposed and evaluated CNNs for intrusion detection systems with KDD99 and NSL-KDD datasets, respectively. The first one achieved up to 99.23% accuracy compared to other algorithms, such as support vector machine and deep belief network. The second work achieved a better accuracy, e.g., 97%, than other techniques, such as recurrent neural network (RNN), long short-term memory (LSTM), and gated recurrent units (GRU).

The related work still leaves gaps for research on developing CNN for FPGA. Some prior work focuses solely on using a CPU or FPGA to implement CNNs. In contrast, others aim to achieve high accuracy by comparing results obtained with different machine learning algorithms. However, these approaches do not prioritize performance and energy efficiency.

Our work differs from others in proposing a CNN architecture on a CPU+FPGA hybrid multichip package with high bandwidth to exploit performance with energy efficiency. Following task partitioning concepts, we designed execution flows to harness the power of highly parallel execution units on the FPGA for convolution operations. This hybrid multichip package is a standard system in next-generation firewalls, enabling flexibility and the deployment of ahead-of-the-curve techniques.[10,11] Furthermore, implementations of CNN using OpenCL for FPGA-based multichip packages and intrusion detection are still few explored. Thus, our main advancement to the related work is an energy-efficient CPU+FPGA-based CNN architecture for network intrusion detection.

PROPOSED CNN ARCHITECTURE

Our hardware architecture was designed to support floating-point arithmetic to maintain consistency with the software-based CNN implementation, which also used floating-point arithmetic. For this reason, this article does not exploit power consumption savings from quantization due to the potential loss in accuracy.

Although FPGAs offer high energy efficiency, they are limited by finite memory and logic blocks. Additionally, implementing an entire CNN on an FPGA can be challenging and time-consuming, requiring extensive optimization and tuning to achieve high performance and energy efficiency. Therefore, we decided to implement only the convolution operation on the FPGA. This operation is computationally intensive and benefits the most from FPGA acceleration. By offloading this operation to the FPGA, we achieved significant performance gains and energy efficiency improvements without overloading the FPGA with additional computations. Thus, we achieved an efficient and

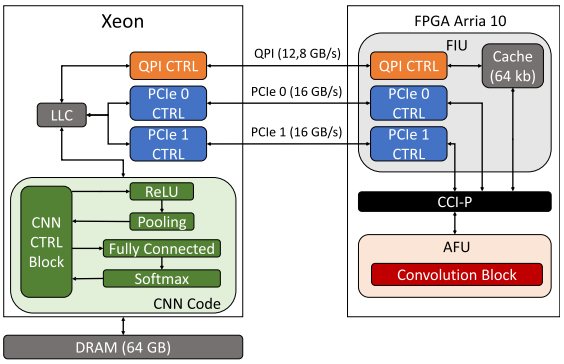

FIGURE 2. CPU+FPGA CNN architecture.

scalable CNN implementation that leveraged the CPU and FPGA's strengths.

Our CPU+FPGA-based CNN architecture was designed using OpenCL as a high-level synthesis (HLS) strategy, which provides a standard programming interface for heterogeneous computing systems. This approach allows us to develop portable code easily deployed on different hardware platforms, including hybrid CPU+FPGA systems. Additionally, we used the Tiny-DNN deep learning library-DNN[25] to construct our custom neural network. With the C++ and OpenCL approaches, the host (CPU) starts the FPGA and its objects and memory units and synchronizes the operations performed in parallel, using work items at the kernel level. This implementation approach allowed us to adopt a rationale centered around highly parallel execution units for the convolution operations on the FPGA. Using OpenCL, these execution units can be efficiently designed, such as through the use of indexed work items, and organized within an *N*-dimensional range (NDRange). This NDRange enables the execution of kernel instances for each work item, facilitating parallel processing.

The CPU+FPGA architecture (Figure 2) is based on the following interconnected blocks: 1) the Xeon is responsible for containing the connection interfaces with the FPGA, with the DRAM memory, and storing the CNN programming logic; 2) the CNN code includes the modules responsible for executing the functions of the neural network and controlling the operations carried out in each flow of the algorithm, whether steps in the CPU or activating the device; 3) the FPGA-FIU module has the bitstream responsible for making the interface between the interconnections and the bitstream provided by the CPU; 4) the FPGA-CCI-P interface exposes communication channels to CPU; 5) and the FPGA-AFU module represents the convolution step of the CNN algorithm that will be executed on the FPGA.

Our design receives a previously trained CNN and a dataset of test records as input for evaluation and simulation of the detection of network intrusions. As output, the class identified for each record and its accuracy is available.

Figure 1 shows a CNN architecture, receiving 121-feature input processed by two convolution layers, two pooling layers, and three fully connected layers. The last layer is responsible for classifying the network intrusion records into six types of classes. The size of convolution layer kernels is [5*5] and [2*2], and the size of pooling layers is [2*2]. In addition, the architecture uses fully connected layers with 22 neurons in the first two layers and six neurons in the last. The CNN uses the rectified linear unit (ReLU) activation function between each layer block, except in the last one, where it applies the Softmax function.[26]

The size of the network layers was defined initially considering the number of types of attacks (22) and attributes (144 or 11×11) that would be evaluated. Based on this information, changes were made to the CNN hyperparameters to assess which one would have more acceptable accuracy.

The CNN operation (see Figure 3) starts instantiating FPGA memories and loading data structures to store the records it will process. After, the previously trained model is loaded.

The CNN processes items in layers, executing only those related to convolution on the FPGA. The reason is to achieve high performance since the operations related to matrix multiplication are highly parallelizable and demand greater computational power. In addition, we also applied the NDRange strategy, allowing the FPGA to use several computing units with high power of parallelism in each available neuron.

The architecture sends processed records back to the host. Next, the host runs the other layers of *pooling*, *ReLU*, and *fully-connected*, using parallel loop structures, improving processing power. Finally, the system informs the

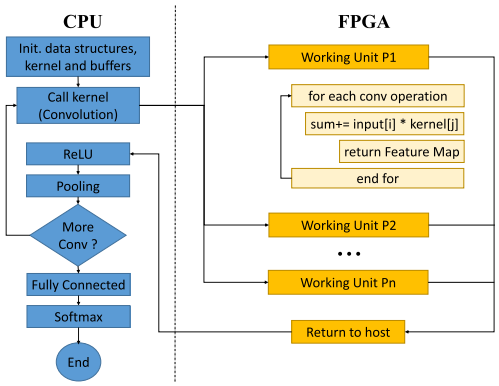

FIGURE 3. CNN architecture flow.

user of the final classification of the evaluated record and these steps are carried out iteratively for each existing record in the dataset. The accuracy of the neural network is verified only at the end of the entire process, comparing the original label of the record with the one classified by the network.

EVALUATION METHODOLOGY

We compared our CPU+FPGA OpenCL-based architecture with CPU-only OpenMP-based software version with 24 threads, using the same input data. We used an Intel Xeon E5-2620 processor with 2.4 GHz. The number of threads chosen is related to the best results achieved. Our heterogeneous architecture proposal runs on the Intel hybrid multichip package, with a 2.4 GHz Intel Xeon CPU + Arria 10 GX1150 FPGA, with a frequency of up to 400 MHz and 65.6 MB of internal memory, allowing high data transfer.

The CNN used two samples of NSL-KDD records[24] for training and testing. In addition, we developed a script in C language for reading a CSV file with 41 columns, where 40 are attributes and 1 is a record label. Their values have been normalized between 0-1, using the min–max technique.[22] The strategy converts the columns with categorical values (*protocol_type*, *service*, *flag*) into new columns with binary values. For instance, *protocol_type* has three types of attributes: TCP, UDP, and ICMP, which form three columns [1,0,0], [0,1,0], and [0,0,1]. Thus, at the end of the process, we will have 122 columns, i.e., 121 attributes and one label. Finally, six classes group 22 types of attacks according to their characteristics (*Dos*, *U2r*, *R2l*, *Probe*, *Normal*, and *Unknown*).

Initially, we trained the CNN using a software version with 125,000 input records and 64 iterations/epochs to generate a model with 90.59% accuracy for the CPU-Only and CPU+FPGA platforms.

With the model trained, we tested the CNN, varying the number of records according to the Fibonacci sequence times one thousand (1000, 2000, 3000, 5000, 8000, 13,000, and 21,000) due to the 22,543 records limit in the NSL-KDD dataset.

To compare the software and CPU+FPGA approaches, we evaluated the execution time and energy consumption. The Intel PowerPlay EPE returns the FPGA approach's power consumption. Meanwhile, the PowerTOP[27] tool does the same for the Xeon processor. We calculated the energy efficiency metric as the ratio between each workload's total number of operations and the consumed power. In other words, the metric is called millions of floating point operations per second (MFLOPS) per watt, i.e., MFLOPS/W.

RESULTS

This section highlights our main results related to FPGA occupation and energy efficiency. The multichip package consists of a device with two units: the FPGA interface unit (FIU) and the accelerated function unit (AFU). Table 1 shows the device occupation as the number of logical elements, adaptive lookup tables (ALUTs), registers, memory blocks, and digital signal processing (DSP) blocks. Arria 10 remains a large number of available resources for other potential kernels.

For a comprehensive CNN analysis, Figures 4, 5, and 6 show execution time, energy

TABLE 1. FPGA occupation.

Resources	FIU+CCI	AFU
Logical Elements	563 (49%)	58 (5%)
ALUTs	98,256 (23%)	4272 (1%)
Registers	461,376 (27%)	68,352 (4%)
Memory Blocks	15,466 (23%)	4035 (6%)
DSP Blocks	182 (12%)	15 (1%)

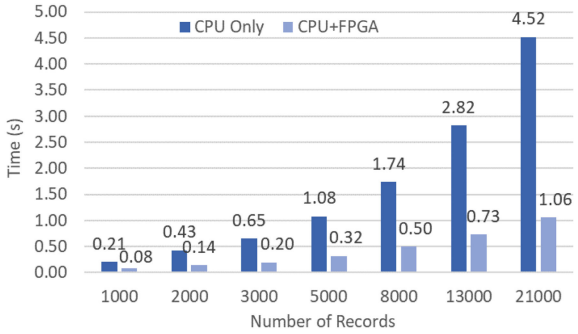

FIGURE 4. Execution time (seconds).

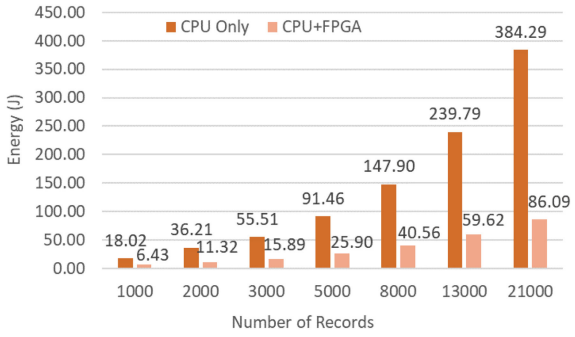

FIGURE 5. Energy consumption (Joules).

consumption, and energy efficiency results for CPU+FPGA and CPU-only platforms. Thus, we executed 24 threads, with a variation of records (Rec) between 1000 and 21,000. The execution time to process several network intrusion records by the CPU+FPGA approach is up to 77% lower than the CPU-Only version. In addition, there is a growing difference in the results as the number of records increases, which shows better scalability of the heterogeneous platform when working with large workloads due to the FPGA as an accelerator. The energy consumption shows that the CPU+FPGA platform consumes up to 78% less than the CPU-Only software version.

In terms of energy efficiency, measured in millions of floating-point operations per second (MFLOPS) per watt, the heterogeneous platform can run up to 4.5× more MFLOPS/watt than the CPU-only software version. The execution time impacts the energy results in both scenarios, even considering the average consumption of Xeon at 85 W and CPU+FPGA at 81.45 W.

Our results show that we can implement CNN architectures on CPU+FPGA heterogeneous

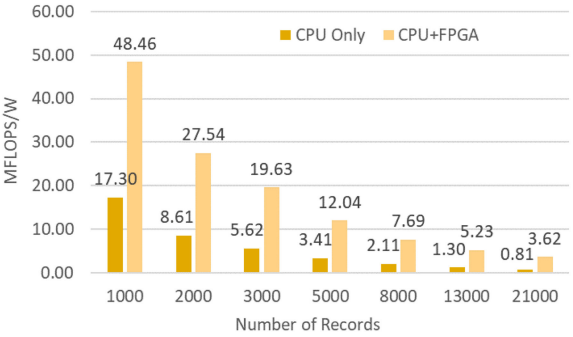

FIGURE 6. Millions of floating-point operations per second per watt (MFLOPS/W).

multichip packages for network intrusion detection systems. They can meet scenarios with varied types and amounts of data to promptly identify anomalies with energy efficiency.

CONCLUSION

This work presented a heterogeneous CNN architecture for intrusion detection using OpenCL and C/C++ languages on the Intel HARPv2 platform (Intel Xeon + Arria 10). We implemented a CNN using the C++ Open-Source library called Tiny-DNN. This CNN version using parallel approaches achieved better performance on CPU+FPGA heterogeneous platform than the CPU-only version of up to 77% in execution time, up to 78% in energy consumption and up to 4.5× in MFLOPS/watt. For future work, one can adopt memory sharing and load distribution strategies to explore a higher level of heterogeneity for more efficiency and scalability. Besides, we intend to compare this current high-level synthesis OpenCL-based CPU+FPGA solution with other heterogeneous architectures, e.g., CPU+GPU, focusing on flexibility, portability, performance, and efficiency. Finally, regarding the potential tradeoff between processing and accuracy performances, our planning includes an FPGA design space exploration to be efficient in execution time, energy consumption, and accuracy for different neural network algorithms.

ACKNOWLEDGMENTS

This work was supported by CAPES (Code 001), CNPq, FAPEMIG, and Intel HARP.

REFERENCES

1. M. A. Souza, L. A. Maciel, P. H. Penna, and H. C. Freitas, "Energy efficient parallel K-means clustering for an Intel hybrid multi-chip package," in *Proc. 30th Int. Symp. Comput. Architecture High Perform. Comput.*, 2018, pp. 372–379, doi: 10.1109/CAHPC.2018.8645850.
2. L. A. Maciel, M. A. Souza, and H. C. Freitas, "Reconfigurable FPGA-Based K-means/K-Modes architecture for network intrusion detection," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 67, no. 8, pp. 1459–1463, Aug. 2020, doi: 10.1109/TCSII.2019.2939826.
3. M. O. Farooq, I. Wheelock, and D. Pesch, "IoT-Connect: An interoperability framework for smart home communication protocols," *IEEE Consum. Electron. Mag.*, vol. 9, no. 1, pp. 22–29, Jan. 2020, doi: 10.1109/MCE.2019.2941393.
4. I. Ahmed, G. Jeon, and A. Ahmad, "Deep learning-based intrusion detection system for Internet of Vehicles," *IEEE Consum. Electron. Mag.*, vol. 12, no. 1, pp. 117–123, Jan. 2023, doi: 10.1109/MCE.2021.3139170.
5. M. S. Krishna and T. Perumal, "Making buildings smarter and energy-efficient—Using the Internet of Things platform," *IEEE Consum. Elect. Mag.*, vol. 10, no. 3, pp. 34–41, May 2021, doi: 10.1109/MCE.2021.3053182.
6. A. Halimaa A. and K. Sundarakantham, "Machine learning based intrusion detection system," in *Proc. 3rd Int. Conf. Trends Electron. Inform.*, 2019, pp. 916–920, doi: 10.1109/ICOEI.2019.8862784.
7. Y. Li, "Research on application of convolutional neural network in intrusion detection," in *Proc. 7th Int. Forum Elect. Eng. Autom.*, 2020, pp. 720–723, doi: 10.1109/IFEEA51475.2020.00153.
8. E. Chen et al., "Application of improved convolutional neural network in image classification," in *Proc. Int. Conf. Mach. Learn., Big Data Bus. Intell.*, 2019, pp. 109–113, doi: 10.1109/MLBDBI48998.2019.00027.
9. J. Li, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-Based energy-efficient reconfigurable convolutional neural network accelerator for object recognition applications," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 68, no. 9, pp. 3143–3147, Sep. 2021, doi: 10.1109/TCSII.2021.3095283.
10. M. S. Brunella et al., "HXDP: Efficient software packet processing on FPGA NICs," *Commun. ACM*, vol. 65, no. 8, pp. 92–100, Aug. 2022, doi: 10.1145/3543668.
11. J. Lázaro et al., "Embedded firewall for on-chip bus transactions," *Comput. Elect. Eng.*, vol. 98, 2022, Art. no. 107707, doi: 10.1016/j.compeleceng.2022.107707.
12. Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Comput. Commun.*, vol. 198, pp. 175–185, 2023, doi: 10.1016/j.comcom.2022.11.001.
13. D. Wang, K. Xu, and D. Jiang, "PipeCNN: An OpenCL-based open-source FPGA accelerator for convolution neural networks," in *Proc. Int. Conf. Field Programmable Technol.*, 2017, pp. 279–282, doi: 10.1109/FPT.2017.8280160.
14. M. R. Vemparala, A. Frickenstein, and W. Stechele, "An efficient FPGA accelerator design for optimized CNNs using OpenCL," in *Proc. Int. Conf. Archit. Comput. Syst.*, 2019, pp. 236–249, doi: 10.1007/978-3-030-18656-2_18.
15. J. Jiang, M. Jiang, J. Zhang, and F. Dong, "A CPU-FPGA heterogeneous acceleration system for scene text detection network," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 69, no. 6, pp. 2947–2951, Jun. 2022, doi: 10.1109/TCSII.2022.3167022.
16. L. Ioannou and S. A. Fahmy, "Network intrusion detection using neural networks on FPGA SoCs," in *Proc. 29th Int. Conf. Field Programmable Log. Appl.*, 2019, pp. 232–238, doi: 10.1109/FPL.2019.00043.
17. D.-M. Ngo, A. Temko, C. C. Murphy, and E. Popovici, "FPGA hardware acceleration framework for anomaly-based intrusion detection system in IoT," in *Proc. 31st Int. Conf. Field Programmable Log. Appl.*, 2021, pp. 69–75, doi: 10.1109/FPL53798.2021.00020.
18. L. Zhang, X. Yan, and D. Ma, "Accelerating in-vehicle network intrusion detection system using binarized neural network," *Int. J. Adv. Curr. Pract. Mobility*, vol. 4, no. 6, pp 2037–2050, 2022, doi: 10.4271/2022-01-0156.
19. L. Le Jeune, T. Goedemé, and N. Mentens, "Towards real-time deep learning-based network intrusion detection on FPGA," in *Proc. Appl. Cryptogr. Netw. Secur. Workshops*, 2021, pp. 133–150, doi: 10.1007/978-3-030-81645-2_9.
20. L. L. Jeune et al., "SoK - Network Intrusion Detection on FPGA," in *Proc. Int. Conf. Secur., Privacy, Appl. Cryptogr. Eng.*, 2022, pp. 242–261, doi: 10.1007/978-3-030-95085-9_13.
21. R. U. Khan et al., "An improved convolutional neural network model for intrusion detection in networks," in *Proc. Cybersecurity Cyberforensics Conf.*, 2019, pp. 74–77, doi: 10.1109/CCC.2019.000-6.

22. Y. Ding and Y. Zhai, "Intrusion detection system for NSL-KDD dataset using convolutional neural networks," in *Proc. ACM Int. Conf. Comput. Sci. Artif. Intell.*, 2018, pp. 81–85, doi: 10.1145/3297156.3297230.
23. S. Al-Emadi, A. Al-Mohannadi, and F. Al-Senaid, "Using deep learning techniques for network intrusion detection," in *Proc. IEEE Int. Conf. Inform., IoT, Enabling Technol.*, 2020, pp. 171–176, doi: 10.1109/ICIoT48696.2020.9089524.
24. L. C. Hong, "NSL-KDD dataset," 2015. [Online]. Available: https://github.com/defcom17/NSL_KDD
25. E. Riba, "Tiny-DNN," 2016. [Online]. Available: https://github.com/tiny-dnn/tiny-dnn
26. M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. USENIX Conf. Operating Syst. Des. Implementation*, 2020, pp. 265–283, doi: 10.5555/3026877.3026899.
27. Intel, "PowerTOP," 2007. [Online]. Available: https://01.org/powertop

Lucas Andrade Maciel is an IT Engineer Lead with Localiza&Co and an adjunct professor with the Pontifical Catholic University of Minas Gerais (PUC Minas), Belo Horizonte, 30535-901, Brazil. His research interests include computer architecture, high-performance computing, and intrusion detection systems. He received the M.Sc. degree in informatics from PUC Minas in 2020. Contact him at lucasmaciel@pucminas.br.

Matheus Alcântara Souza is an assistant professor with the Pontifical Catholic University of Minas Gerais (PUC Minas), Belo Horizonte, 30535-901, Brazil. His research interests include computer architecture and high-performance computing. He received Ph.D. degree in informatics from PUC Minas in 2021. Contact him at matheusalcantara@pucminas.br.

Henrique Cota de Freitas is an associate professor with the Pontifical Catholic University of Minas Gerais (PUC Minas), Belo Horizonte, 30535-901, Brazil. His research interests include computer architecture and high-performance computing. He received the Ph.D. degree in computer science from the Federal University of Rio Grande do Sul, Porto Alegre, Brazil, in 2009. He is a member of the IEEE. Contact him at hcfreitas@ieee.org.